Step 4 — Production Wrap (optional) — Solution

> Back to the step · Hint 1 · Hint 2

Story: this wraps the finished 4-agent system in Project 5's production layer — a real FastAPI `/review` endpoint, plus an eval suite that specifically checks the routing logic — proving that production layer generalizes to a new agent team too, not just the one it was first built for. **If not:** CodeGuard would stay a thing you describe in an interview instead of a real link you can actually hand someone to try.

This step reuses Project 5 almost unchanged. No new ideas — just repetition.

`src/pipeline.py` — the one door

```
# src/pipeline.py
# The single entry point the API and the terminal both call: build the graph,
# run it on a diff, return the compiled review.
from src.graph import build_graph
from src.state import start_state

_APP = None


def get_app(model, style_collection):
    global _APP
    if _APP is None:
        _APP = build_graph(model, style_collection)
    return _APP


def review_diff(model, style_collection, diff):
    app = get_app(model, style_collection)
    final = app.invoke(start_state(diff))
    return {
        "review": final["review"],
        "reviewers_run": final["routing_log"],
        "finding_count": len(final["findings"]),
    }
```

The FastAPI layer

```
# src/api/schemas.py
from pydantic import BaseModel


class ReviewRequest(BaseModel):
    diff: str


class ReviewResponse(BaseModel):
    review: str
    reviewers_run: list
    finding_count: int
```

```
# src/api/routes.py
from fastapi import APIRouter

from src.api.schemas import ReviewResponse
from src.config import load_config
from src.models.llm import get_model
from src.pipeline import review_diff
from src.rag.ingest import build_style_store
from src.rag.store import get_collection

router = APIRouter()
_CONFIG = load_config()
_MODEL = get_model(_CONFIG)
_COLLECTION = build_style_store(get_collection())

@router.get("/health")
def health():
    return {"status": "ok"}

@router.post("/review", response_model=ReviewResponse)
def review(request):
    result = review_diff(_MODEL, _COLLECTION, request.diff)
    return ReviewResponse(**result)
```

```
# src/api/main.py
# uvicorn src.api.main:app --host 0.0.0.0 --port 8000
from fastapi import FastAPI

from src.api.routes import router
```

```
app = FastAPI(title="CodeGuard")
app.include_router(router)
```

```
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "src.api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

The routing eval (the one that matters)

A suite that only checks each reviewer's accuracy could pass even if routing were completely broken. So the key eval checks **routing**: a docs-only diff must not produce a Security finding, and must not run the Security reviewer.

```
# src/evals/routing_eval.py
# Prove the routing works, not just the reviewers. Run from the repo root:
#   python -m src.evals.routing_eval
from src.config import load_config
from src.models.llm import get_model
from src.pipeline import review_diff
from src.rag.ingest import build_style_store
from src.rag.store import get_collection

DOCS_ONLY = "--- a/README.md\n+++ b/README.md\n+a new line in the docs\n"
CODE_WITH_SECRET = (
    "--- a/app.py\n+++ b/app.py\n+api_key = \"SUPERsecret12345\"\n"
)


def main():
    config = load_config()
    model = get_model(config)
    collection = build_style_store(get_collection())

    docs = review_diff(model, collection, DOCS_ONLY)
    ok_docs = "security" not in docs["reviewers_run"]
    verdict_docs = "FAIL"
    if ok_docs:
        verdict_docs = "PASS"
```

```
    print(f"routing (docs-only skips security) -> {verdict_docs}")

    code = review_diff(model, collection, CODE_WITH_SECRET)
    ok_code = "security" in code["reviewers_run"] and "secret" in code["review"]
    verdict_code = "FAIL"
    if ok_code:
        verdict_code = "PASS"
    print(f"routing (code runs security + flags secret) -> {verdict_code}")


if __name__ == "__main__":
    main()
```

The API test (team faked, no key)

```
# tests/api/test_routes.py
# The API with the pipeline faked, so it runs with no model and no key.
#   python -m pytest tests/api/test_routes.py -v
import os

os.environ.setdefault("OPENAI_API_KEY", "sk-test")


def test_review_route(monkeypatch):
    from src import pipeline

    def fake_review(model, collection, diff):
        return {"review": "CodeGuard review: [HIGH] (security) secret",
                "reviewers_run": ["security", "coverage", "style"],
                "finding_count": 1}

    monkeypatch.setattr(pipeline, "review_diff", fake_review)
    from fastapi.testclient import TestClient
    from src.api.main import app
    client = TestClient(app)
    assert client.get("/health").json()["status"] == "ok"
    response = client.post("/review", json={"diff": "x"})
    assert response.status_code == 200
    assert "security" in response.json()["review"]
```

Expected routing-eval output (real model):

```
routing (docs-only skips security) -> PASS
```

```
routing (code runs security + flags secret) -> PASS
```

That's the production wrap: the same FastAPI + eval + Docker layer from Project 5, on a completely different system — the final proof that both halves of what you learned, the agent design *and* the production layer, generalize. With this, CodeGuard is a real, runnable service you can hand an interviewer a link to.

The whole project, in one sentence

A Supervisor-led team of three specialist reviewers — Security (a secret/unsafe-pattern scanner), Style (grounded in your real style guide through RAG), and Test-Coverage — routed by what actually changed, compiled into one de-duplicated, severity-sorted review, and served behind a FastAPI endpoint. The same patterns as ContentForge, proven on an entirely different problem.